

Team 7 - E-Waste Battery Extraction

CV-17 Code & Team Handover Guide

Pixel-to-World Localisation Integration

ROS 2 Jazzy / Gazebo / Team 7 Perception Workspace

Ubuntu 24.04 on WSL2

Implementation and handover reference

September 2026

Contents

1 Purpose	2
2 Files Added or Modified	2
3 What Changed from the Original Geometry	2
4 Geometry Summary	3
5 Required Camera Values	3
6 Required TF	4
7 Recommended Launch Order	4
8 Detection-Node Integration	5
9 Important Current Limitation	5
10 Part B Validation Tools	6
10.1 Manual Click Validator	6
10.2 Automatic Green Validator	6
11 Validated Performance	7
12 Common Checks and Troubleshooting	7
12.1 No TF Available	7
12.2 No Camera Image Arrives	8
12.3 Geometry Module Appears Out of Date	8
12.4 World Coordinates Look Completely Wrong	8
13 Safe Shutdown	8
14 Team Handover Checklist	9
15 Final Handover Status	9

1 Purpose

This guide is the practical handover for the completed CV-17 pixel-to-world localisation work. It is intended for a teammate who needs to run, inspect, reuse or extend the implementation without repeating the full development process.

The final localisation path is:

$$(u, v) \rightarrow \text{CameraInfo} \rightarrow \text{TF} \rightarrow \text{ray-plane intersection} \rightarrow (X, Y, Z)_{\text{world}}.$$

The geometry has been validated independently against Gazebo ground truth at five positions, with a mean planar error of 1.731 mm and a maximum error of 2.205 mm.

2 Files Added or Modified

The Part A integration affected these package files:

```
team7_perception_ws/src/team7_perception/
|-- launch/
| '-- localisation_tf.launch.py [added]
|-- team7_perception/
| |-- geometry.py [modified]
| '-- onnx_phone_detector_node.py [modified]
|-- setup.py [modified]
'-- package.xml [modified]
```

Part B added separate validation utilities in the workspace root:

```
cv17_validate_click.py
cv17_validate_green.py
cv17_validation_auto_results.csv
cv17_validation_summary.csv
cv17_validation_evidence/
```

Development backup files such as `geometry.py.bak` and `onnx_phone_detector_node.py.bak` should not be shared as final package files.

3 What Changed from the Original Geometry

The original implementation embedded a fixed camera pose and manually encoded the optical-to-world axis mapping. The final implementation removes the hard-coded camera world position from the projection function.

The active interface is:

```
pixel_to_world_tf(
    u, v,
    fx, fy, cx, cy,
    plane_z,
    camera_pos,
    camera_quat
)
```

Inputs:

Table 1: Inputs to `pixel_to_world_tf`.

Argument	Meaning
<code>u, v</code>	Image-space centroid in pixels
<code>fx, fy, cx, cy</code>	Camera intrinsics from <code>CameraInfo</code>
<code>plane_z</code>	Horizontal target plane height in world coordinates
<code>camera_pos</code>	Optical-frame origin expressed in the world frame
<code>camera_quat</code>	Quaternion for world $\leftarrow$ camera optical frame

Return value:

```
(world_x, world_y, world_z, valid)
```

4 Geometry Summary

The optical ray is:

$$\mathbf{d}_c = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}. \quad (1)$$

The TF quaternion rotates the ray into world coordinates:

$$\mathbf{d}_w = R_{wc} \mathbf{d}_c. \quad (2)$$

For camera origin $\mathbf{C}$, the ray is:

$$\mathbf{P}(t) = \mathbf{C} + t\mathbf{d}_w. \quad (3)$$

Intersecting with a horizontal plane $Z = Z_p$ gives:

$$t = \frac{Z_p - C_z}{d_{w,z}}. \quad (4)$$

The method returns invalid if the ray is parallel to the plane or if the intersection lies behind the camera.

5 Required Camera Values

The Gazebo overhead camera currently publishes:

```
/top_camera/image
/top_camera/camera_info
```

Verified calibration:

```
width = 1280
height = 960
fx = 1108.7654113769531
```

```
fy = 1108.7654113769531
cx = 640.0
cy = 480.0
```

All simulated distortion coefficients are zero.

```
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ ros2 topic echo /top_camera/camera_info --once
header:
  stamp:
    sec: 352
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
height: 960
width: 1280
distortion_model: plumb_bob
d:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
k:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 1.0
r:
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
- 0.0
- 0.0
- 0.0
```

Figure 1: CameraInfo values used by the localisation implementation.

6 Required TF

The localisation code expects:

```
world <- top_camera_optical_frame
```

The current verified transform is approximately:

```
Translation: [0.000, 0.600, 1.900]
RPY (degree): [-180.000, 0.000, -90.000]
```

Start it with:

```
cd ~/team7_project_bundle/team7_perception_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch team7_perception localisation_tf.launch.py
```

Verify with:

```
ros2 run tf2_ros tf2_echo world top_camera_optical_frame
```

7 Recommended Launch Order

Use four WSL terminals.

Terminal 1 - Gazebo

```
bash ~/team7_project_bundle/run_team7_sim.sh
```

Leave the simulation running.

Terminal 2 - Localisation TF

```
cd ~/team7_project_bundle/team7_perception_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch team7_perception localisation_tf.launch.py
```

Terminal 3 - Perception

```
cd ~/team7_project_bundle/team7_perception_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
bash ~/team7_project_bundle/run_team7_perception.sh
```

Terminal 4 - Checks and Validation

```
cd ~/team7_project_bundle/team7_perception_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

8 Detection-Node Integration

The ONNX detector node now reads camera intrinsics, creates a TF listener, looks up the optical-frame transform, and calls `pixel_to_world_tf` for each selected detection. The custom output message contains:

```
world_x
world_y
world_z
world_valid
```

A valid geometry result therefore reaches the existing `/phone_detections` interface without requiring a new downstream message type.

9 Important Current Limitation

Do not use the current live YOLO battery centroid as evidence of geometry accuracy. During CV-17 testing, the detector produced an oversized battery bounding box:

```
bbox_x1: 880
bbox_y1: 19
bbox_x2: 1278
bbox_y2: 911
mar_center_x: 1078.5
mar_center_y: 464.5
```

The world result generated from that centroid is therefore not the physical location of `battery_1`. This is an upstream detection/preprocessing problem and should be corrected separately.

```
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ timeout 10 ros2 topic echo /phone_detections --once
header:
  stamp:
    sec: 696
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
detections:
- header:
  stamp:
    sec: 696
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
  class_id: battery
  score: 0.9073662757873535
  bbox_x1: 880
  bbox_y1: 19
  bbox_x2: 1278
  bbox_y2: 911
  mar_center_x: 1078.5
  mar_center_y: 464.49993896484375
  mar_width: 396.99993896484375
  mar_height: 890.9998779296875
  angle_deg: 90.0
  world_x: 0.015796911411798856
  world_y: 0.15310204041736597
  world_z: 0.77
  world_valid: true
---
```

Figure 2: Known detector-output issue. Geometry validation was intentionally performed independently of this centroid.

10 Part B Validation Tools

10.1 Manual Click Validator

`cv17_validate_click.py` subscribes to the live image, lets the user click an image point, predicts the world coordinate, reads Gazebo ground truth, calculates error and appends the measurement to CSV. It is useful for debugging and demonstrations, but not recommended for formal statistics because click placement is subjective.

10.2 Automatic Green Validator

`cv17_validate_green.py` is the recommended reproducible validation tool. It:

- isolates the green battery in HSV;
- cleans the mask with morphology;
- selects the largest green contour;
- computes a minimum-area rectangle centre;
- calls the actual package geometry function;
- reads live Gazebo ground truth only after prediction;
- stores the result in `cv17_validation_auto_results.csv`;
- saves an annotated evidence image.

Run it with:

```
python3 cv17_validate_green.py
```

The image wait timeout was increased to 180 seconds because the WSL/Gazebo camera stream can temporarily pause even while `camera_info` continues to publish.

11 Validated Performance

The five formal test positions produced:

Table 2: Final CV-17 planar localisation errors.

Position	XY error (mm)
P1	1.55
P2	2.10
P3	2.20
P4	1.51
P5	1.28

Final aggregate statistics:

```
Mean XY error : 1.731 mm
RMSE : 1.767 mm
Minimum error : 1.281 mm
Maximum error : 2.205 mm
Mean dX bias : -0.107 mm
Mean dY bias : +0.846 mm
```

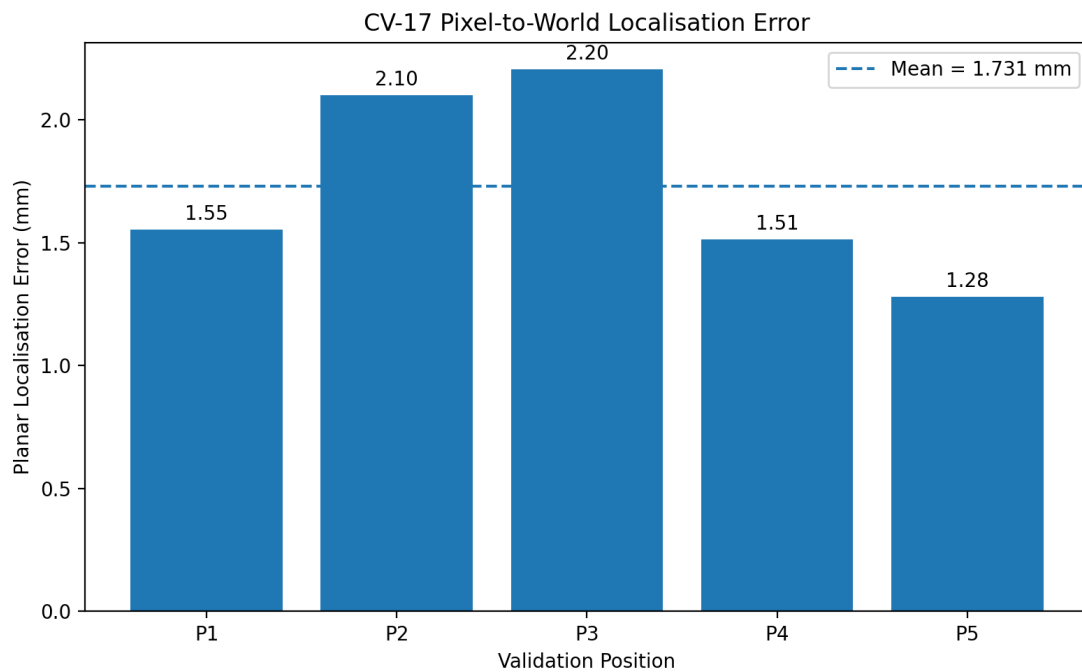


Figure 3: Formal five-position localisation error result.

12 Common Checks and Troubleshooting

12.1 No TF Available

Check that Terminal 2 is still running and verify:

```
ros2 run tf2_ros tf2_echo world top_camera_optical_frame
```

The first **Invalid frame ID** line can occur while the listener starts. If valid transform output immediately follows, no fix is required.

12.2 No Camera Image Arrives

Check:

```
ros2 topic list | grep top_camera
timeout 6 ros2 topic hz /top_camera/image
timeout 6 ros2 topic hz /top_camera/camera_info
```

If the image topic exists but temporarily has no frames, allow the validator to wait. Do not restart the full simulation immediately.

12.3 Geometry Module Appears Out of Date

Inspect the active import:

```
python3 - <<'PY'
import inspect
import team7_perception.geometry as g
print(g.__file__)
print(inspect.signature(g.pixel_to_world_tf))
PY
```

If source files were changed, rebuild the workspace:

```
colcon build --symlink-install
source install/setup.bash
```

12.4 World Coordinates Look Completely Wrong

Before changing the geometry, inspect the image centroid and bounding box. A bad detector centroid can produce a mathematically valid but physically wrong world coordinate. Confirm the geometry independently with a known image point or the automatic validation tool.

13 Safe Shutdown

Recommended shutdown order:

1. stop the perception node with **Ctrl+C**;
2. stop the localisation TF with **Ctrl+C**;
3. stop the simulation using the supplied script.

```
bash ~/team7_project_bundle/team7_sim_mark2_v1.0/scripts/stop_sim.sh
```

14 Team Handover Checklist

Before sharing CV-17 changes with another team member:

- include the final `geometry.py`;
- include `localisation_tf.launch.py`;
- include the updated detector node;
- include the updated `setup.py` and `package.xml`;
- exclude temporary `.bak` files;
- tell the receiver to rebuild with `colcon build -symlink-install`;
- confirm the expected camera topics and TF frame names;
- state the plane-height convention used by their application;
- warn that the current YOLO battery box is a separate upstream issue;
- keep the validation CSV and plots as regression evidence.

15 Final Handover Status

The geometry, TF integration and controlled Gazebo validation are complete. A teammate can reuse the localisation code as long as the correct camera intrinsics, optical-frame TF and physical plane height are supplied. The next integration priority is to correct or replace the current detector centroid association, then test the full detector-to-world pipeline end to end.